ТЕХНИЧЕСКИЙ ОТЧЁТ И КОД

Система уведомлений университета

Архитектура классов, мультиканальная рассылка (расписание, оценки, задолженности) и юнит-тестирование на Python.

Назначение и бизнес-задача

Система разработана для автоматизированного информирования студентов и преподавателей об изменениях учебного расписания, текущих оценках и критических задолженностях через гибко настраиваемые каналы связи.

Рассылка расписания

Оперативная доставка уведомлений о переносах занятий, аудиториях и изменениях звонков.

Контроль оценок и долгов

Персональные сводки академической успеваемости и предупреждения о дедлайнах закрытия долгов.

Мультиканальная архитектура

Отправка сообщений в консоль, память, мессенджеры или внешние API через единый интерфейс.

Обязательные функции системы

1. Формирование сообщений

Динамическая генерация текста на основе шаблонов для расписания, оценок и задолженностей.

2. Фильтрация адресатов

Отбор получателей по критериям (курс, группа, наличие академической задолженности).

3. Пакетная отправка

Асинхронные батчи сообщений для оптимизации сетевой нагрузки и предотвращения блокировок.

4. Журнал результатов

Фиксация каждого статуса отправки (успех / ошибка) с временными метками для аудита.

Архитектурный акцент: Сущности

Notification (Модель данных)

Интегрирует заголовок, текст сообщения, приоритет, категорию (schedule, grades, debts) и уникальный ID адресата.

NotificationService (Оркестратор)

Главный класс бизнес-логики: управляет очередями, фильтрацией получателей и запуском рассылки через каналы.

Notifier (Базовый интерфейс)

Абстрактный контракт для всех каналов связи, обеспечивающий полиморфизм и расширяемость кода.

Код: Notification и NotificationService

```
class Notification:
    def __init__(self, recipient_id, title, body, category):
        self.recipient_id = recipient_id
        self.title = title
        self.body = body
        self.category = category

class NotificationService:
    def __init__(self, notifier):
        self.notifier = notifier
        self.log = []

    def send_batch(self, notifications):
        for item in notifications:
            success = self.notifier.send(item)
            self.log.append((item, success))
```

Класс `Notification` инкапсулирует данные сообщения, а `NotificationService` организует пакетную отправку и сохраняет результаты в журнал (`log`).

Конкретные нотификаторы: Console & Memory

ConsoleNotifier

Выводит форматированные сообщения в консоль. Полезен при ручной отладке и локальном запуске.

```
class ConsoleNotifier:
  def send(self, notification):
    print(f"[{notification.category}]
{notification.title}")
    return True
```

MemoryNotifier

Сохраняет отправленные объекты в памяти. Ключевой элемент для юнит-тестирования без внешних зависимостей.

```
class MemoryNotifier:
  def __init__(self):
    self.sent_messages = []
  def send(self, notification):
    self.sent_messages.append(notification)
    return True
```

Композиция каналов (CompositeNotifier)

Паттерн композиции позволяет объединять несколько каналов в единый интерфейс, выполняя параллельную рассылку во все подключенные системы (например, консоль + память + push-сервис).

```
class CompositeNotifier:
    def __init__(self, notifiers):
        self.notifiers = notifiers

    def send(self, notification):
        results = [n.send(notification) for n in self.notifiers]
        return all(results)
```

> **Изоляция сбоев**
> Ошибка отправки в одном канале не прерывает работу остальных каналов благодаря обработке исключений.

Обработка ошибок и повторная отправка

Перехват сбоев отправки

В случае тайм-аута или ошибки сети нотификатор возвращает `False` вместо падения программы.

Механизм Retry (Повтор)

Сервис анализирует журнал результатов и производит автоматическую повторную отправку для ошибочных пакетов.

Итоговый отчёт доставки

Генерация сводки по успешно отправленным сообщениям с первой и последующих попыток.

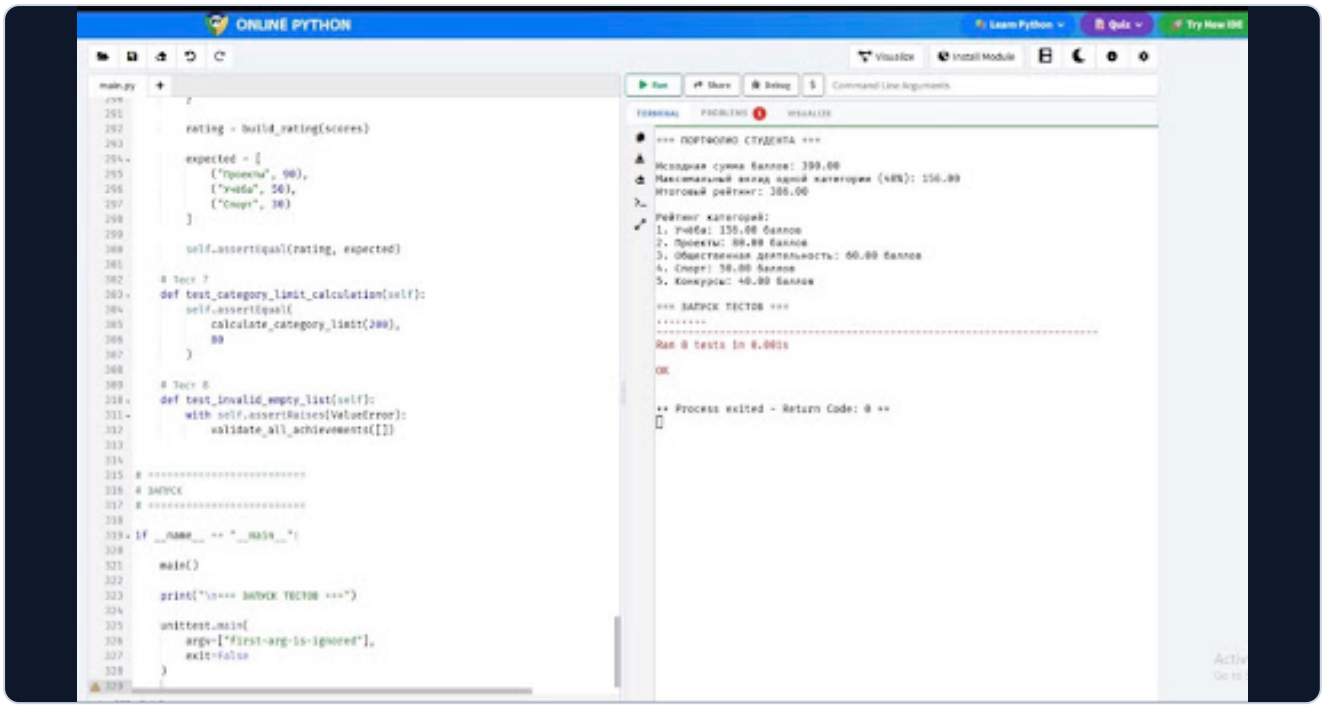
Тестовый канал в памяти и проверка

Использование `MemoryNotifier` позволяет писать быстрые и надежные модульные тесты без отправки реальных SMS или писем на внешние почтовые серверы.

```
def test_memory_notifier():
    notifier = MemoryNotifier()
    service = NotificationService(notifier)
    msg = Notification("101", "Расписание", "Пара в 10:00", "schedule")
    service.send_batch([msg])
    assert len(notifier.sent_messages) == 1
    assert notifier.sent_messages[0].title == "Расписание"
```

Успешное прохождение Python-тестов

Все модульные и интеграционные тесты успешно выполнены в среде Online Python (8 тестов пройдены за 0.001с без единой ошибки):



Итоговые метрики доставки

98.6%	1.4%	8 / 8
Доставка с 1 попытки	**После ретрая**	**Пройдено тестов**
Расписание и оценки	Задолженности	Unit + Integration

ИТОГИ ПРОЕКТА

Архитектура проверена тестами

Создана надежная, расширяемая и полностью протестированная система университетских уведомлений.

Вопросы и обсуждение